Stack Applications: Well-Formedness of Parentheses

Q. Explain how a stack can be used to check the well-formedness of parentheses in an expression. Write a step-by-step algorithmic approach.

> **Definition to Remember**

> An

bracke

closing

---

1. Why Stack Works for This Problem

- LIFO ensures the **most recently opened bracket** is at the top of the stack.

- Wh

2. Algorithm

```

1. Initia

## 3. Trace Example: `{ [ ( ) ] }`

| Character | Action | Stack State |

| :--- | :

**Invalid Example: `{ ( ] }`**

- Rea

---

> **Must-Write Points (for 10 marks)**

> 1. St
```

> **Quick Recall**

> `Ope
O(n)`

C Code: Parentheses Checking Algorithm using Stack

```c

#includ

#define MAX 100

struct CharStack {

char it

void push(struct CharStack *s, char c) {

s->iten

char pop(struct CharStack *s) {

if (s->t

int isMatchingPair(char char1, char char2) {

if (cha

int checkBalanced(char exp[]) {

struct

for (int i = 0; i < strlen(exp); i++) {

if (exp
```

```
int main() {
char e
printf("Expression: %s -> %s\n", expr1, checkBalanced(expr1) ? "Balanced " : "Unbalanced ");
printf("
return 0;
}
```